DECK 2 OF 3 | SUDO HARDENING --START

$ LINUX SERVER SECURITY HARDENING

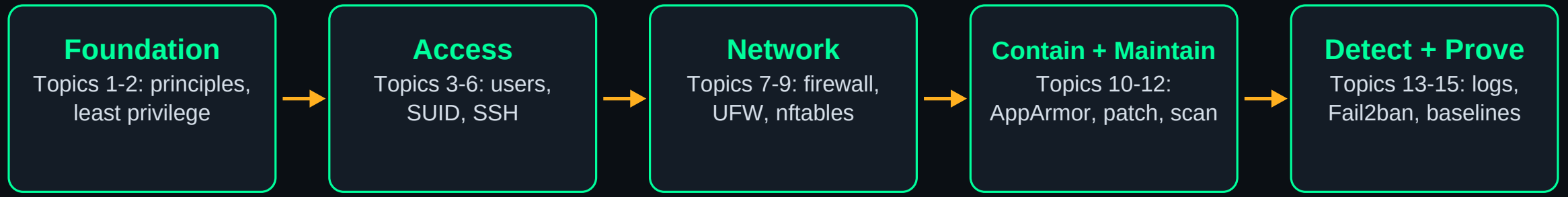
Reduce the attack surface, step by step

15 Topics | 5 Layers | Hands-on commands

Presented by: Tyrone Justine A. Medina

root@server:~# hardening --roadmap

$ Roadmap: 15 Topics, 5 Layers



- Think of building a house: foundation, doors, fence, alarm, inspection
- Each topic ends with commands you can run and a way to verify the result
- Always keep a second SSH session open while you change access or firewall rules

root@server:~# hardening --topic 01

$ 1. Security Principles: the CIA Triad

Confidentiality

Only authorized people can read data. Controls: file permissions, encryption, strong authentication.

Integrity

Data is not changed without permission. Controls: file integrity monitoring, signed packages, backups.

Availability

Systems work when needed. Controls: patching, redundancy, rate limiting, capacity planning.

> *Defense in Depth: stack several layers (Data > Host > Network > Perimeter) so one failure is not a breach.*

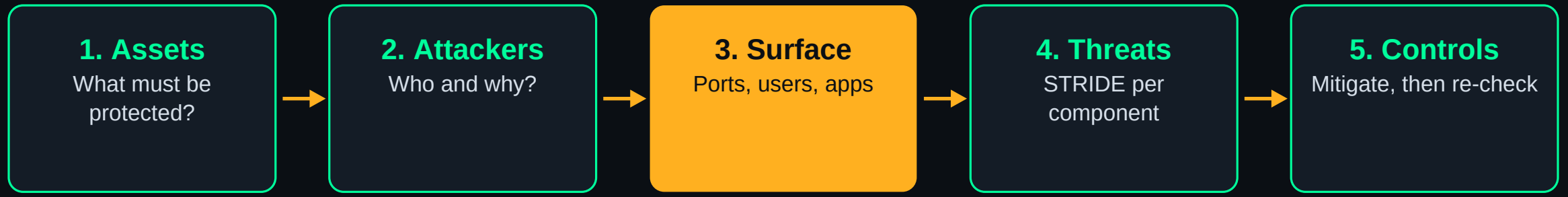
root@server:~# hardening --topic 01

$ 1. Threat Modeling with STRIDE

Threat	Property broken	Example	Typical control
Spoofing	Authentication	Stolen SSH password	Keys, MFA
Tampering	Integrity	Edited config or binary	Permissions, FIM
Repudiation	Non-repudiation	Deleted logs, shared accounts	auditd, remote logs
Info disclosure	Confidentiality	World-readable DB backup	Permissions, encryption
Denial of service	Availability	SYN flood, full disk	Rate limits, quotas
Elevation of privilege	Authorization	Vulnerable SUID binary	Least privilege, patching

root@server:~# hardening --topic 01

$ 1. Threat Modeling in 5 Steps



- **Risk = Likelihood x Impact**
  rank the findings, fix the highest first
- **Example**
  SSH open to the internet: likelihood high, impact high, so it goes to the top of the list
- Revisit the model whenever you add a service, a user group or a network path

root@server:~# hardening --topic 02

$ 2. Principle of Least Privilege

LEAST PRIVILEGE

```
# Create an admin user and grant sudo
sudo adduser deploy
sudo usermod -aG sudo deploy

# Limited: may only restart nginx
sudo visudo -f /etc/sudoers.d/deploy
deploy ALL=(ALL) /bin/systemctl restart nginx

# Service account that cannot log in
sudo useradd -r -s /usr/sbin/nologin svc-app

# Review sudo rights
sudo -l -U deploy
```

- **PoLP**
  give only the minimum access needed
- **No direct root login**
  use sudo, it logs who did what
- **Service accounts**
  nologin shell
- Run services as a non-root user

root@server:~# hardening --topic 02

$ 2. Sudo Hygiene and Service Sandboxing

SUDOERS DEFAULTS

```
# /etc/sudoers.d/00-hardening
Defaults use_pty
Defaults logfile="/var/log/sudo.log"
Defaults timestamp_timeout=5

# Validate before you leave
sudo visudo -c
sudo chmod 0440 /etc/sudoers.d/*

# Who used sudo?
sudo grep sudo /var/log/auth.log | tail
```

SYSTEMD SANDBOX

```
# /etc/systemd/system/myapp.service
[Service]
User=svc-app
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true
ReadWritePaths=/var/lib/myapp

# Score the result (lower is better)
systemd-analyze security myapp
```

> *A bad sudoers file can lock out every admin. Always edit with visudo and keep a root session open.*

root@server:~# hardening --topic 03

$ 3. User and Group Security Policies

USERS AND GROUPS

```
# Password aging
sudo chage -M 90 -m 7 -W 14 alice
sudo chage -l alice

# Lock / expire unused accounts
sudo usermod -L olduser
sudo usermod -e 2025-01-01 olduser

# Find non-root UID 0 accounts (backdoor?)
awk -F: '$3==0 {print $1}' /etc/passwd

# Find empty passwords
sudo awk -F: '$2==""  {print $1}' /etc/shadow

# Group-based access
sudo groupadd devops && sudo usermod -aG devops alice
```

- **UID 0 audit**
  UID 0 equals root. A common backdoor.
- **Groups**
  easier to audit than per-user permissions
- **Aging**
  chage forces rotation and warns users first
- Review accounts every quarter and when people leave

root@server:~# hardening --topic 03

$ 3. Password Quality and Lockout

PAM_PWQUALITY

```
sudo apt install libpam-pwquality

# /etc/security/pwquality.conf
minlen = 12
minclass = 3
maxrepeat = 3
dictcheck = 1

# Test a candidate password
pwscore
```

PAM_FAILLOCK

```
# /etc/security/faillock.conf
deny = 5
fail_interval = 900
unlock_time = 900

# Inspect and reset
faillock --user alice
faillock --user alice --reset
```

> *pam_faillock must also be added to the PAM stack (see man pam_faillock). Test it on a non-admin account first.*

root@server:~# hardening --topic 04

$ 4. SUID, SGID and Sticky Bit

SUID (4xxx)

Runs as the file owner. Example: /usr/bin/passwd runs as root so users can change their password.

SGID (2xxx)

Runs as the group. On directories, new files inherit the directory group.

Sticky (1xxx)

Only the owner can delete files in the directory. Example: /tmp.

> *Unexpected SUID files (in /tmp, /home or a custom path) are a classic privilege-escalation route. Investigate each one.*

root@server:~# hardening --topic 04

$ 4. Finding and Removing Special Permissions

SPECIAL PERMISSIONS

```
# Find all SUID / SGID files
find / -perm -4000 -type f 2>/dev/null
find / -perm -2000 -type f 2>/dev/null

# Remove what is not needed
sudo chmod u-s /path/to/file

# Setting them (for reference)
chmod 4755 file      # SUID
chmod 2775 dir       # SGID
chmod 1777 /shared   # Sticky
```

- Save the list as a baseline and diff it after updates
- Normal SUID examples: passwd, sudo, mount, su
- Anything else needs a reason

root@server:~# hardening --topic 04

$ 4. Permission Audit

FILE PERMISSIONS

```
# World-writable files and directories
find / -xdev -type f -perm -0002 -ls
find / -xdev -type d -perm -0002 ! -perm -1000 -ls

# Files with no owner or group
find / -xdev \( -nouser -o -nogroup \) -ls

# Sensitive files
ls -l /etc/passwd /etc/shadow /etc/gshadow
sudo chmod 640 /etc/shadow

# Safer default for new files
grep ^UMASK /etc/login.defs      # set UMASK 027
```

- **World-writable**
  anyone can modify the file
- **No owner**
  often left by deleted users, can be re-used by a new UID
- **UMASK 027**
  new files: owner full, group read, others none

root@server:~# hardening --topic 05

$ 5. SSH Key-Based Authentication

- The private key never leaves your laptop, only a signature crosses the network
- Brute force against a key is impractical, unlike a password
- ed25519 keys are short, fast and the current recommended default

root@server:~# hardening --topic 05

$ 5. Creating and Installing Keys

SSH KEYS

```
# 1) Generate a key pair (on the laptop)
ssh-keygen -t ed25519 -C "alice@laptop"

# 2) Install the public key on the server
ssh-copy-id alice@server

# 3) Correct permissions (on the server)
chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys

# 4) Test
ssh alice@server

# Optional: cache the passphrase
eval "$(ssh-agent)" && ssh-add
```

- **Passphrase**
  always set one
- **One key per device**
  revoke a lost laptop without touching others
- **Offboarding**
  remove the key from authorized_keys
- Never copy a private key to a server

root@server:~# hardening --topic 06

$ 6. SSH Server Configuration

SSHD_CONFIG

```
# /etc/ssh/sshd_config
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
PermitEmptyPasswords no
AllowUsers alice bob          # or AllowGroups sshusers
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
ClientAliveInterval 300
ClientAliveCountMax 2
LogLevel VERBOSE

# Test BEFORE reloading!
sudo sshd -t
sudo systemctl reload ssh
```

- **Warning**
  Confirm key login works before disabling passwords, or you may lock yourself out.
- **Keep a session open**
  until the new login is confirmed to work

root@server:~# hardening --topic 06

$ 6. SSH Access Restrictions

MATCH BLOCKS

```
# Per-source rule
Match Address 192.168.1.0/24
    PasswordAuthentication yes

# SFTP-only jail
Match Group sftponly
    ChrootDirectory /srv/sftp
    ForceCommand internal-sftp
    AllowTcpForwarding no
```

KEY OPTIONS + 2FA

```
# authorized_keys (one line)
from="192.168.1.0/24",no-port-forwarding ssh-ed25519 AAAA... alice

# Key + one-time code
sudo apt install libpam-google-authenticator
AuthenticationMethods publickey,keyboard-interactive
```

> *Changing the SSH port only reduces log noise. The real protection is keys, a firewall, rate limiting and monitoring.*

root@server:~# hardening --topic 07

$ 7. Host-Based Firewall: the Idea

- A firewall on the server itself, not only at the network edge
- Default deny: allow only what is needed
- UFW is the simple front-end, nftables runs underneath
- Allow SSH before enabling, or you cut your own session

root@server:~# hardening --topic 08

$ 8. UFW: Uncomplicated Firewall

UFW

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 443/tcp
sudo ufw allow from 192.168.1.0/24 to any port 22
sudo ufw limit ssh
sudo ufw logging on
sudo ufw enable
sudo ufw status verbose
sudo ufw status numbered
sudo ufw delete 3
```

- **limit**
  blocks an IP that opens 6+ connections in 30 seconds
- **numbered**
  delete a rule by its number
- Run status verbose after every change

root@server:~# hardening --topic 08

$ 8. UFW: Profiles, Blocks and Logs

RULES

```
# Name-based app profiles
sudo ufw app list
sudo ufw allow "Nginx Full"

# Block one address
sudo ufw deny from 203.0.113.50

# Only from a subnet, TCP
sudo ufw allow from 192.168.1.0/24 \
  to any port 5432 proto tcp
```

MAINTENANCE

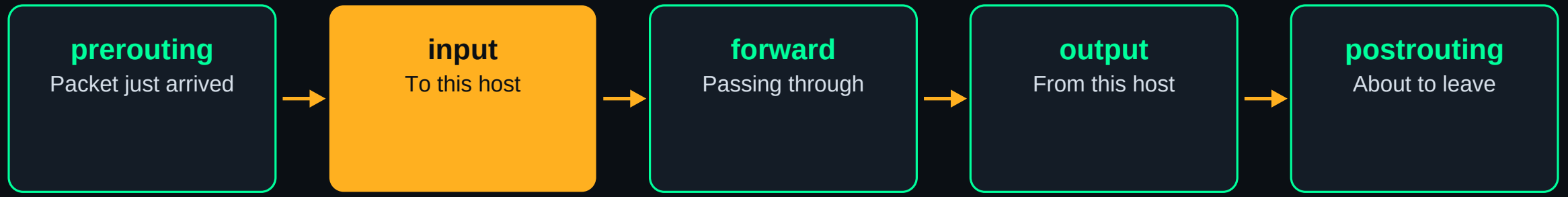
```
# Review, reload, reset
sudo ufw status numbered
sudo ufw delete 3
sudo ufw reload
sudo ufw reset          # wipes all rules

# Logs
sudo ufw logging medium
sudo tail -f /var/log/ufw.log
```

> *Docker publishes ports with its own iptables rules and can bypass UFW. Bind containers to 127.0.0.1 unless they must be public.*

root@server:~# hardening --topic 09

$ 9. Netfilter Hooks



- **Netfilter**
  packet-filtering framework inside the kernel
- **nftables**
  the modern replacement for iptables
- **Structure**
  table > chain > rule, IPv4 and IPv6 in one syntax

root@server:~# hardening --topic 09

$ 9. nftables: a Default-Drop Ruleset

NFTABLES

```
# /etc/nftables.conf
table inet filter {
  chain input {
    type filter hook input priority 0; policy drop;
    ct state established,related accept
    ct state invalid drop
    iif lo accept
    meta l4proto ipv6-icmp accept
    icmp type echo-request limit rate 5/second accept
    tcp dport 22 ct state new limit rate 10/minute accept
    tcp dport { 80, 443 } accept
  }
  chain forward { type filter hook forward priority 0; policy drop; }
}

sudo nft -c -f /etc/nftables.conf    # syntax check
sudo systemctl enable --now nftables
sudo nft list ruleset
```

root@server:~# hardening --topic 09

$ 9. iptables vs nftables

Aspect	iptables (legacy)	nftables
Status	Legacy, still common	Current default on modern Debian and Ubuntu
Tools	iptables, ip6tables, arptables	One tool: nft, family inet covers IPv4 + IPv6
Performance	Rules checked one by one	Sets and maps for fast lookups
Reload	Rule-by-rule changes	Atomic ruleset replacement
Migration	iptables-save	iptables-translate, iptables-restore-translate

> *Pick one firewall front-end (UFW or nftables) per server. Mixing them leads to rules nobody can explain.*

root@server:~# hardening --topic 10

$ 10. AppArmor: Enforce vs Complain

Enforce mode

- Blocks violations and logs them
- Use in production
- sudo aa-enforce /usr/sbin/nginx

Complain mode

- Only logs, does not block
- Use while building or refining a profile
- sudo aa-complain /usr/sbin/nginx

> *MAC: each program has its own profile in /etc/apparmor.d/. Even if nginx is compromised, its reach is limited.*

root@server:~# hardening --topic 10

$ 10. AppArmor Commands and Profile

COMMANDS

```
sudo aa-status
sudo apt install apparmor-utils

sudo aa-complain /usr/sbin/nginx
sudo aa-enforce /usr/sbin/nginx

sudo aa-genprof /usr/bin/myapp
sudo aa-logprof

sudo apparmor_parser -r \
  /etc/apparmor.d/usr.sbin.nginx
sudo journalctl -k | grep -i apparmor
```

SMALL PROFILE

```
#include <tunables/global>
/usr/bin/myapp {
  #include <abstractions/base>
  /etc/myapp/** r,
  /var/log/myapp/*.log w,
  /var/lib/myapp/** rw,
  deny /etc/shadow r,
}
```

root@server:~# hardening --topic 11

$ 11. Patch Management

UPDATES

```
sudo apt update && sudo apt upgrade
apt list --upgradable
sudo apt autoremove

# Preview without changing anything
apt-get -s upgrade

# Security status (Ubuntu)
ubuntu-security-status
```

AUTOMATIC + RESTARTS

```
sudo apt install unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades
sudo unattended-upgrades --dry-run --debug

# Is a reboot / restart needed?
cat /var/run/reboot-required
sudo needrestart
```

> *Snapshot first. Patch a test host before production. A patch that is never restarted into memory does not protect you.*

root@server:~# hardening --topic 12

$ 12. Vulnerability Scanning

SCANNING

```
# Port / service discovery
sudo nmap -sV -p- localhost

# Host audit
sudo apt install lynis
sudo lynis audit system

# Full scanner: OpenVAS / Greenbone
# web console, scans a target range, ranks by CVSS
```

- **CVSS**
  Critical 9.0-10, High 7.0-8.9, Medium 4.0-6.9, Low below 4
- **Priority**
  CVSS score plus exposure
- **Cycle**
  Test, Stage, Deploy, Verify
- Fix High/Critical on internet-exposed services first

root@server:~# hardening --topic 13

$ 13. Authentication Logs

LOGS

```
# Logs
sudo tail -f /var/log/auth.log      # RHEL: /var/log/secure
sudo journalctl -u ssh
last -n 10                          # successful logins
sudo lastb -n 10                    # failed logins

# Top attacker IPs
grep "Failed password" /var/log/auth.log | \
  awk '{print $(NF-3)}' | sort | uniq -c | sort -nr | head
```

- **Tip**
  Forward logs to another server or SIEM. Attackers delete local logs first.
- Check failed logins daily until you have alerting (Deck 3)

root@server:~# hardening --topic 13

$ 13. Auditing with auditd

QUICK USE

```
sudo apt install auditd
sudo systemctl enable --now auditd

# Live watch (lost on reboot)
sudo auditctl -w /etc/sudoers -p wa -k sudoers

# Search by key, readable output
sudo ausearch -k sudoers -i
sudo ausearch -m USER_LOGIN -sv no -i
```

PERSISTENT RULES

```
# /etc/audit/rules.d/hardening.rules
-w /etc/passwd -p wa -k identity
-w /etc/shadow -p wa -k identity
-w /etc/ssh/sshd_config -p wa -k sshd
-a always,exit -F arch=b64 -S execve -F euid=0 -k rootcmd

sudo augenrules --load
sudo auditctl -l
sudo aureport --auth --summary
```

> *-p wa means watch for writes and attribute changes. -k adds a key so you can search quickly.*

root@server:~# hardening --topic 14

$ 14. Brute-Force Defense with Fail2ban

FAIL2BAN

```
# /etc/fail2ban/jail.local
[DEFAULT]
ignoreip = 127.0.0.1/8 192.168.1.0/24
bantime  = 1h
findtime = 10m
maxretry = 5

[sshd]
enabled = true

[recidive]
enabled = true
bantime = 1w

sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd
sudo fail2ban-client set sshd unbanip 203.0.113.50
```

- **How it works**
  reads logs, bans IPs that fail too often using the firewall
- **ignoreip**
  whitelist your own networks so you do not ban yourself
- **recidive**
  longer bans for repeat offenders

root@server:~# hardening --topic 15

$ 15. Baselines and Hardening Verification

VERIFY

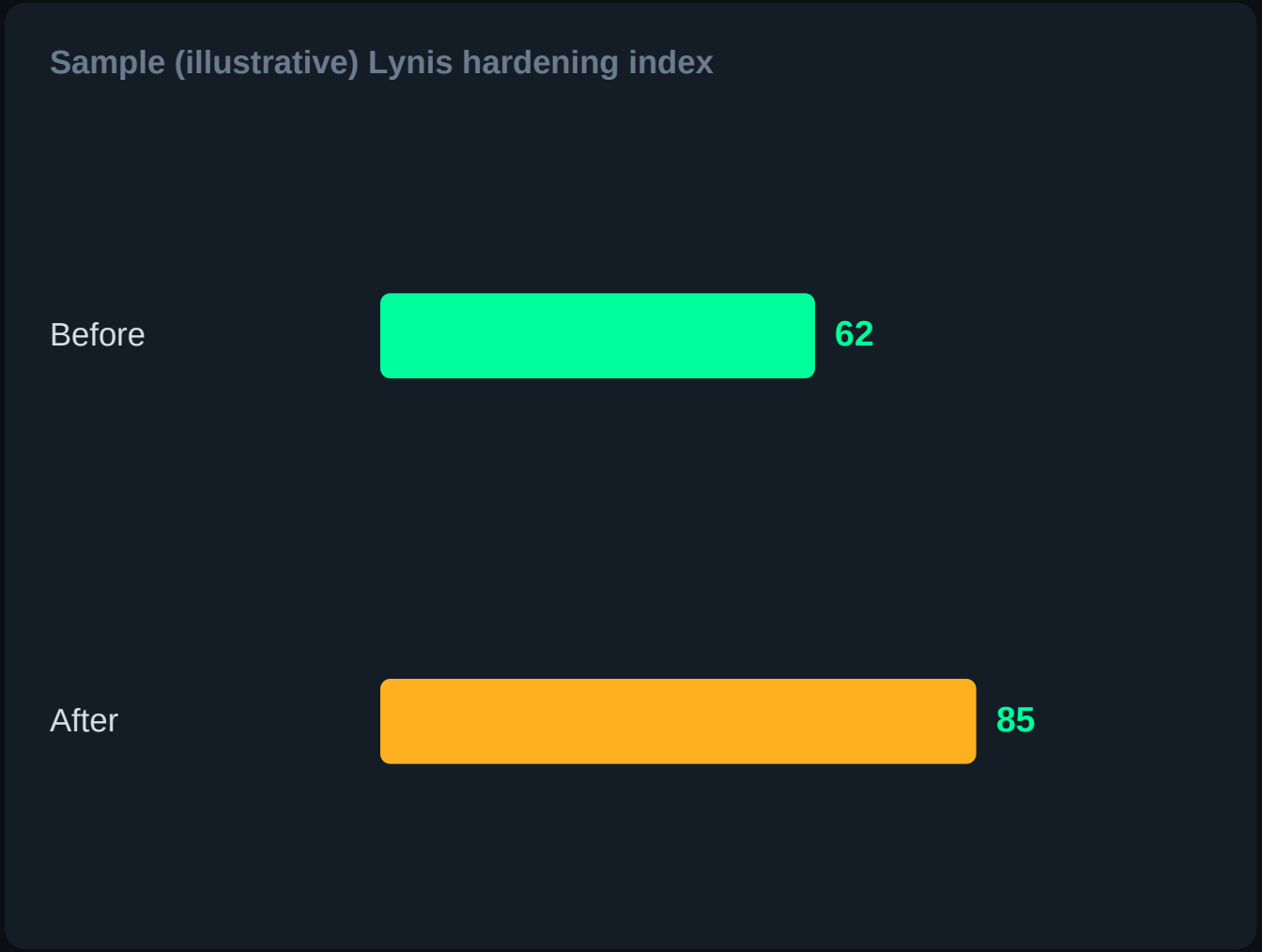
```
# Lynis
sudo lynis audit system
sudo grep -E "hardening_index|suggestion" \
  /var/log/lynis-report.dat

# OpenSCAP (CIS profile)
sudo oscap xccdf eval --profile \
  xccdf_org.ssgproject.content_profile_cis \
  --report report.html \
  /usr/share/xml/scap/ssg/content/ssg-ubuntu2204-ds.xml
```

- **Baseline**
  CIS Benchmarks, DISA STIG
- **Repeat**
  after every major change
- Open report.html and fix failed rules by priority

root@server:~# hardening --topic 15

$ 15. Measuring Progress



- Record the score before you start
- Re-scan after each layer and track the trend
- A higher score is a signal, not proof of security
- **Security = layers + habit**
  repeat the cycle

hardening complete [OK]

$ Summary: Hardening Checklist

- **Think**
  threat model and CIA
- **Limit**
  least privilege, user policies, SUID audit
- **Lock**
  SSH keys, UFW/nftables, AppArmor
- **Maintain**
  patching and scanning
- **Watch**
  logs, auditd, Fail2ban
- **Verify**
  CIS baseline and Lynis

Q & A

Next: Deck 3, Security Monitoring with Wazuh, Snort and Zeek